ARTCRAFT FLEET TECHNICAL BROCHURE · SERIES #2

GameCraft v0.14.0

Describe a game in a sentence. Get a Godot 4 project you can open, play, and edit.



v0.14.0	10 Crates	37735	6-Gate
RELEASE VERSION	WORKSPACE TIERS	PURE-RUST LOC	ARTCRAFT VERIFIED

Repository: `D:\Dev\repos\gamecraft` · **Standard:** `CRAFT_BROCHURE_STANDARD.md` · **Compiled via:** `BookCraft v0.5.0`

Describe a game in a sentence. Get a Godot 4 project you can open, play, and edit.

```
gamecraft-cli new --prompt "a snowy mountain pass at dusk, hard jumps" --provider local
```

That runs a language model (a local Ollama model by default) which writes a small JSON description of a game: sky colours, jump strength, where the platforms and coins go, how far the goal is. GameCraft validates that description, compiles it into a real Godot scene, boots Godot headless, and checks the game can actually be finished. If it can't, you get a failure, not a broken level.

The output is a normal Godot project folder. Open `project.godot` in the editor, press F5, or hit Start Game in the desktop viewer.

Godot is one of three tracks in the same library: **Godot** (`depot/games/`, compiled from specs and booted headless), **three.js browser games** (`web/games/`, served locally and verified headless), and **Bevy sims** (`bevy/games/`, native executables). 16 + 19 + 3 and counting. The prompt path (`new`) targets Godot only.

What you get

`new` writes a minimal but complete Godot project:

```
depot/games/snowy-mountain-pass/
  project.godot      # open this in Godot 4.4
  main.tscn          # the generated scene
  spec.json          # the model's description, the input to the compiler
  scripts/           # player, camera, runtime
  icon.svg
```

The hand-built depot games additionally carry `lint.json`, `trace.json`, `commands.jsonl`, `thumb.png`, `play.gif` and a `README.md`. The prompt path does **not** produce those yet - no thumbnail, no recorded replay, no per-game lint dump. That gap is real and not hidden here.

What the AI actually controls

Dial	Effect
`theme`	six colours: sky, horizon, ground, platforms, player, coins
`physics`	run speed, jump impulse, gravity - this is the "game feel"
`level`	ground slabs, floating platforms, coin positions, goal distance
`maze`	size (odd 7-25) and seed - a deterministic perfect maze
`title` / `slug`	name and folder

A full example is committed at `depot/games/snowy-mountain-pass/spec.json`.

What it cannot do yet

Straight answers, because this is the part that matters:

- **Eight archetypes, not thirteen.** Prompts can produce `platformer`, `runner`, `pong`, `maze`, `flight`, `driving`, `chess` and `talk`. Ask for a shooter and the compiler rejects the spec and lists what is supported - it does not silently hand back a platformer.
- **What each archetype lets you change:** `platformer` gets the full level (ground slabs, platforms, coins, goal) plus physics and theme; `runner` gets theme and name (its jump timing is a fixed verified script); `pong` gets theme, name and ball speed (8-22); `maze` gets size (odd 7-25), seed and name; `flight` gets gate count, spacing and altitude; `driving` gets loop radius and checkpoint count; `chess` gets board size, hop count and seed; `talk` gets how many conversations the door requires.
- **The other genres exist but are hand-built.** The 16 depot games span platformer, runner, flight, driving, pong, breakout, flappy, maze, sokoban, sokoban-vault, chessboard, dialogue, shooter and tower defence. Only the eight above are wired to the prompt path. Flappy stays unwired on purpose: it is one of the four that never proved an in-engine win, so generating it would ship games I cannot verify.
- **Local model quality varies.** `gemma4:12b` works. `qwen3.8:27b` crashes Ollama outright (socket reset, no output in 420 s) - do not use it.
- **Cloud providers are wired but only tested against a stub.** No live API key was available, so OpenAI/Anthropic paths are unproven end to end.

Why "spatial compiler"

The name describes the plumbing, not the pitch.

A language model asked to "write a game" will invent function names that do not exist and produce GDScript that fails on line 3. So the model here never writes code. It writes data. GameCraft then:

1. **Validates** the data (supported archetype, positive sizes, reachable goal).
2. **Compiles** it deterministically into Godot scene text - same spec, same bytes, every run.
3. **Verifies** the result: lint the scene, replay the level in Rust, then boot Godot and require the win line to print.

A hallucinating model can therefore produce a *rejected spec*. It cannot produce a broken scene, because the compiler is the only thing that writes scenes.

Is the output actually any good?

Checked, not assumed. `just verify-depot` boots every Godot depot game in Godot and requires its win line:

```
11 of the long-standing 15 games win
4 unproven: breakout, flappy, topdown-shooter, tower-place
sokoban-vault (added 2026-11-07): not gated yet
```

All 15 boot with zero engine errors. The four that do not win are documented with their specific cause in the Depot section - they are broken designs, not unverified guesses, and the gate stays red to say so.

The demo games

`depot/games/` (16 Godot projects), `web/games/` (19 browser games) and `bevy/games/` (maze, murmuration, toy-railway) are the proof, not the product - examples of what the pipelines emit, each with a trace recording or a verify proof and something playable. The interesting use is generating your own.

Quickstart

```
$env:Path = "$env:USERPROFILE\.cargo\bin;$env:Path"

# gates
cargo fmt --all -- --check
cargo clippy --workspace --all-targets --all-features -- -D warnings
cargo test --workspace
cargo check -p gamecraft-core --target wasm32-unknown-unknown

# make a game from a prompt (local Ollama by default)
cargo run -p gamecraft-cli -- new --prompt "night forest, hard jumps"

# or skip the model entirely and compile a spec you wrote
cargo run -p gamecraft-cli -- new --from-spec depot/games/snowy-mountain-pass/spec.json

# check every depot game still boots and wins
just depot-godot
just verify-depot

# viewer
cargo run -p gamecraft-desktop

# web arcade (three.js family: serve, then open http://127.0.0.1:11258/)
cd web; bun install; bun run serve

# MCP server over stdio
cargo run -p gamecraft-mcp
```

`new` needs neither Godot nor a GPU: it compiles and lints the scene in Rust and replays the level in the Rust sim. You need **Godot 4.4** on `PATH` (or `GAMECRAFT_GODOT`) to play the result or to run the depot gates, and **Ollama** with a model pulled for `--provider local`. `--provider auto` probes local servers first. `--from-spec` skips the model entirely.

Controls

In the Godot games: arrow keys or WASD to move, Space/Enter to jump.

Browser games have per-game keys - see the game's `CHEATSHEET.md` (linked from its detail card).

Gamepad. Godot's built-in `ui_*` actions carry joypad bindings for movement (D-pad and left stick) but `ui_accept` ships keys-only, so a pad could walk and never jump. The shared runtime adds the missing buttons at startup:

Input	Effect
D-pad / left stick	move, walk, drive, chess cursor
A	jump, flap, confirm knight move
B	quit the game (there is no menu to return to)

Desktop viewer (`gilrs`, polled per frame): left stick orbits the camera, D-pad or stick vertical moves the depot selection, **A** starts the highlighted game, **B** returns selection to the scene root. Keyboard and mouse still work; a missing pad degrades to "not connected".

Settings

One config file feeds every entry point: `%USERPROFILE%\.gamecraft\config.json` (`~/.gamecraft/config.json`). The CLI reads it, the viewer edits it through Settings > App settings. Precedence for every field: **CLI flag > config file > environment variable > built-in default.**

Field	What it does	Default
`provider`	`auto` / `local` / `openai` / `anthropic` / `deepseek` / `meta`	`auto`
`model`	model id; empty = auto-pick (gemma4 family); presets in the dialog	empty
`ollama_endpoint`	local server URL; empty = provider default	empty
`openai_api_key`	cloud key; empty = `OPENAI_API_KEY` env	empty
`anthropic_api_key`	cloud key; empty = `ANTHROPIC_API_KEY` env	empty
`deepseek_api_key`	cloud key; empty = `DEEPSEEK_API_KEY` env	empty
`meta_api_key`	cloud key; empty = `META_API_KEY` env	empty
`godot_binary`	override for launching games	empty = `godot`

Keys are never printed or logged; the file inherits your profile's permissions.

Per-game tuning lives next to each game as `settings.json` (gitignored - local tuning, not content). Settings > Game settings writes it for the highlighted game; Save & play launches immediately. The shared runtime reads it at startup:

```
{"speed_mult": 1.2, "gravity_mult": 1.0, "jump_mult": 1.1, "autoplay": null}
```

Multipliers scale the baked-in base values, so one schema fits all sixteen games. `autoplay: null` means "whatever the environment says" - the verifier and the demo button win over the file. Backgrounds and level geometry are baked in and still need regeneration.

Depot

`depot/games/` holds 15 standalone Godot 4.4 projects. Each carries `project.godot`, `main.tscn`, `scripts/`, `lint.json`, `trace.json`, `commands.jsonl`, `thumb.png`, `play.gif` and a `README.md`; the two produced from a spec also carry `spec.json`.

Three gates per game, enforced by the forge:

1. `scene_lint` reports 0 findings (collision shapes, signal targets, resource ids, camera).

2. The Rust trace replay reaches the goal within 600 frames, using the same constants the GDScript exports.

3. `godot --headless --path depot/games/<slug> --quit-after 240` prints no ERROR/WARNING.

Plus a byte-identical re-export: the forge builds each game twice and refuses a non-identical scene. **Never hand-edit** `depot/` - regenerate it with `just forge`.

Win verification, and why it is red

Booting is not winning. `just verify-depot` replays each game in Godot and requires the win line its runtime prints (`goal reached`, `puzzle solved`, `door opened, win`, or `all hostiles down`).

11 of 15 win today: driving, flight, night-forest, platformer, runner, snowy-mountain-pass, maze, pong, sokoban, chessboard-move, talk-inventory.

The four that do not are unproven, not broken - all boot with zero errors:

Game	What is missing
`breakout`	The paddle tracks the ball, so every return is dead straight; the ball clears its own column then loops. Bricks are destructible, but 20 is unreachable.
`flappy`	Closed-loop altitude hold added; the bird still dies on pipes before x=55.
`topdown-shooter`	Bullet pool fixed (it was empty - the runtime filtered on invisibility), enemies die, but it stalls at 5 of 9.
`tower-place`	No placement policy; enemies never enter tower range.

Building blocks

Crate	Role		
`gamecraft-core`	Node graph, transforms, AABB, collision matrix. No graphics deps; compiles to `wasm32-unknown-unknown`.		
`gamecraft-commands`	Undoable command stack for every scene mutation.		
`gamecraft-godot`	Deterministic `.tscn` / `.tres` parser **and serializer**, UID hashing, lint rules.		
`gamecraft-sim`	Deterministic gameplay sims: kinematic stepper, BFS maze and crate solvers, and the seeded multi-threaded flock sim behind Murmuration.		
`gamecraft-replay`	`.gcraft` replay documents: game, config (seed and level), input log, recorded result. `verify` re-simulates from scratch and compares status, ticks and a state hash, so edited inputs, a swapped seed or changed rules are caught. Maze and murmuration. Examples in `docs/replays`, gate `just replays-check`.		
`gamecraft-forge`	Game builders, `GameSpec` validation and compilation, the LLM front end, thumbnail rasterizer, economy and Steam asset generators.		
`gamecraft-gltf`	Deterministic glTF 2.0 export (`.glb` or embedded `.gltf`): hierarchy, primitive meshes shared by geometry, PBR colours, lights, camera. Also owns the primitive tessellation the desktop viewport draws. Gate: `just gltf-gate` (Khronos validator + three.js render).		
`gamecraft-trellis`	Image-to-3D prop pipe (TRELLIS.2 sidecar, plan-only without a Linux/CUDA box): L1 types + driver, CLI `trellis plan	assets	execute`, `MCP` asset.trellis`. See` docs/TRELLIS2.md`.

| `gamecraft-bevy` | GameCraft games on Bevy 0.20 on top of `gamecraft-sim`: headless logic plugin plus an optional windowed renderer (`render` feature). Games: a 29x29 maze that plays itself, Murmuration (20 000 starlings, hawks, pillars, a flare), and the Toy Railway (two-level timetabled model railway; `just railway-gate`), all with procedural sound (`audio` feature, clips synthesised in `synth.rs`, `--mute` or `GAMECRAFT_MUTE=1` to silence; gate runs are muted). Catalog entries live in `bevy/games/<slug>/`. The Nekomimi VRM viewer (phase 1:

orbit, ground, path plumbing) lives here too, behind the `vrm` feature. `just sounds` renders every effect to `docs/sounds/` (WAV, spectrogram PNG, loudness table) and `just sounds-check` fails if those files are stale. Gates: `just bevy-gate`, `just murmuration-gate`. |

`gamecraft-mcp`	**MCP stdio server, dual-era (stateless `2026-07-28` with `server/discover`, plus the `initialize` handshake for `2025-11-25` back to `2024-11-05`): scene tools `scene_lint`, `spatial_query`, `scene_inspect`, `export_scene`; catalog tools `game_list`, `game_report`, `game_verify`, `game_create`; opt-in `scene_verify`. Every result is a one-sentence summary plus `structuredContent`; failures are `isError` results that say what to do next. Also hosts `catalog`, the game catalog shared with CLI and desktop. See `docs/MCP_CLIENTS.md`. Also the command registry (`commands [--json]`, `describe <id>`, `run --cmd 'id={json}'`; `id=@file.json` reads params from a file) and MCP tools `list_commands`, `describe_command`, `run_command`, `replay_record`, `replay_verify`, `replay_info` (12 commands in all). No arguments or `mcp` still means stdio. Status against the Craft app contract: `docs/CRAFT_CONTRACT.md`.**
`gamecraft-cli`	`lint`, `query`, `uid`, `forge`, `new`, `export`, `gltf <scene> [--out]` / `gltf --depot-out <dir>`, plus catalog parity `list [--filter] [--json]`, `report <slug> [--json]`, `verify <slug> [--timeout] [--json]`. Also `commands`, `describe`, `run --cmd` (the same registry as the MCP server).
`gamecraft-desktop`	`egui` / `wgpu` app. **Library** mode (default): card grid of all Godot, three.js and Bevy games by category, detail card with description, last gate result, cheat sheet, Play / Watch demo / Run tests / Open folder; `/` filter, arrows, Enter. **Editor** mode: scene tree, shaded viewport, inspector, lint, trace scrubber, undo/redo; the side panels and the depot/scene and inspector/findings splits are draggable (View > Reset layout). Light/dark toggle, Ctrl+wheel UI zoom, golden-PNG tests (`egui_kittest`).

Games are **compiled from a spec, never hand-written in GDScript** - that is what makes the byte-identical depot gate possible.

Monetisation (Phase 8)

`gamecraft-forge::economy` models monetisation as a deterministic state machine and synthesises a store asset bundle. Both are library APIs today; exposing them as CLI subcommands and MCP tools is the next step, not a claim of this release.

- `economy_lint(&EconomySpec)` flags faucet-dominant economies, missing or late scarcity (target is day 4), undecoupled currency tiers, inverted or late pity (past 90 pulls), and a missing $0.99 starter anchor.
- `simulate(&EconomySpec, seed)` - seeded, reproducible day-by-day balances with pity-ramped pulls.
- `write_steam_bundle(slug, title, out)` - 20 files: trading cards, emoticons, profile backgrounds, 4K wallpapers, badge definition, Supporter Edition manifest, cloud-save JSON Schema, soundtrack.

Honest scope note: the soundtrack is `soundtrack.wav` (mono PCM generated in process). Real FLAC needs an encoder this crate does not carry, so the manifest names the file that exists. Wallpapers are generated gradients - placeholder-grade.

Browser games (three.js family)

`web/` is another, deliberately different track. The depot's rule is "the model writes a spec, never code". The web family breaks it: the model writes the whole game as a static three.js page, the way recent frontier models do well. What can still be asserted is asserted:

```
cd web; bun install
just web-serve     # http://127.0.0.1:11258/
just web-verify    # headless boot, zero console errors, self-play must win
```

Nineteen games across six categories (descriptions live in each game's `game.json`; the full playable list is the arcade index itself):

Calm puzzles (bento, gassi, kintsugi, organ-builder, robot-sorting, shadow-sculpture), **Puzzle** (minesweeper, sudoku classic/color/letters), **Board classics** (hnefatafl), **Seek and find** (hoshi, spot-the-difference, tidy-the-room, where-is-benny), **Worlds and sims** (kaffeehaus, lantern-river, mossbrook), **Detective** (detective-case). Every game ships `game.json` plus a headless `verify.json` proof (all `won: true`); detective-case is metadata-only for now - it has no playable page yet.

Player cheat sheets: `web/games/mossbrook/CHEATSHEET.md` (with walkthrough), `web/games/kaffeehaus/CHEATSHEET.md` (every number in the sim). Developer cheat sheet: `web/CHEATSHEET.md`.

The gate proves a game loads, its systems behave along a scripted tour, and it can be finished. It does not prove it is fun. See `web/README.md`.

Where to look

- `AGENTS.md` - repo layout, hard gates, rules.
- `CHANGELOG.md` - history, including the two serializer bugs found and fixed.
- `docs/NEW_GAME.md` - spec format, providers, measured model limits.
- `docs/MCP_CLIENTS.md` - wiring the MCP server into a client.
- `web/README.md`, `web/CHEATSHEET.md` - the three.js family and how to add a game.
- `docs/LLM_AS_GAME_DESIGNER.md` - methods, strengths and weaknesses of a model as game designer, asset generators and libraries, social VR (Overte, Resonite, VRChat), licensing, and a human/model division of labour. Based on this repo.
- `KIBITZ.md` / `KIBITZ_BACKTALK.md` - the review dialogue, kept as an audit trail.

Status — does today / next week / non-goals

Does today: v0.14.0, 10 crates + CLI + desktop + web arcade; 246 test fns; 14 docs files; 16 Godot + 19 web + 3 Bevy games; sentence → verified playable Godot (`new` → `verify` → `report` / `export`).

Next week: `serve` decision (persistent session + snapshot polling); canonical-engine note (Godot-first?); 3 recipes stay green.

Non-goals: not Bevy-and-Godot forever — canonical note pending; no LLM in the verify path.

Lineage — the Storytold craft-app pattern

Storytold ArtCraft (storytold/artcraft — "the IDE for artists") was the starting point. The 16 craft apps in the Storytold stable each ship one binary that is both the human CLI and the agent MCP server (stdio), headless-first, with `run.json` + report artifacts. That modus operandi and harness is what this repo implements; the code here is independent, the pattern is shared.